

ZÁVĚREČNÁ PROJEKTOVÁ DOKUMENTACE

Testování REST API

Postman + DBeaver | Student CRUD API

Projekt ověřuje funkčnost REST rozhraní pro práci se studenty, chybové stavy, a konzistenci dat mezi API a PostgreSQL databází.

Testované rozhraní: <https://test-app.engeto.cz>

Datum testování: 17. 8. 2026

1. Cíl a rozsah projektu

Cílem projektu bylo ručně otestovat REST API pro správu studentů. Testování probíhalo v Postmanu, zatímco DBeaver sloužil pouze pro čtení a ověření, zda se změny provedené přes API správně propadly do databáze. Databáze nebyla přes SQL upravována.

- Autorizace: POST /login a Bearer token.
- CRUD scénáře: POST, GET, PUT a DELETE nad /students.
- Kontrola chybových stavů: neplatný token, chybějící token, duplicitní e-mail.
- SQL kontrola: pouze SELECT dotazy nad public.students.

Dokumentace nálezů: PASS/FAIL, explicitní požadavky na API, SQL kontroly a defect reporty.

Bezpečnost: přihlašovací údaje ani Bearer token nejsou v dokumentu ani v odevzdávané Postman collection zveřejněny.

2. Testovací prostředí a data

Položka	Hodnota
API	https://test-app.engeto.cz
API dokumentace	Swagger UI /docs
API klient	Postman - Lightweight API Client
Databázový klient	DBeaver
Databáze	PostgreSQL, databáze students
Hlavní testovací záznam	student_id = 249
Doplňkový validační záznam	student_id = 250

3. Souhrn provedených testů

ID	Test	Očekávání	Skutečný výsledek	Stav
TC01	POST /login	Platné přihlášení vrátí token.	200 OK, accessToken vrácen.	PASS

TC02	POST /students	Vytvoření nového studenta.	201 Created, vytvořen ID 249.	PASS
TC03	GET /students/249	Existující student je dostupný.	200 OK, data studenta vrácena.	PASS
TC04	SQL SELECT ID 249	Záznam existuje také v DB.	1 řádek, data odpovídají API.	PASS
TC05	PUT /students/249	Aktualizovaná data se uloží beze změny významu.	200 OK, ale isEuCitizen true se uložilo jako false.	FAIL
TC06	GET po PUT	Vrácená data odpovídají PUT requestu.	GET vrací isEuCitizen = false.	FAIL
TC07	SQL po PUT	DB odpovídá odeslaným hodnotám.	DB obsahuje is_eu_citizen = false.	FAIL
TC08	DELETE /students/249	Student je po úspěšném DELETE odstraněn.	200 OK + 'Student deleted successfully'.	FAIL
TC09	GET po DELETE	Smazaný student již není dostupný.	200 OK, student 249 stále vrácen.	FAIL
TC10	SQL po DELETE	Smazaný záznam není v DB.	1 řádek, student 249 stále existuje.	FAIL
TC11	GET bez tokenu	Neautorizovaný požadavek je odmítnut.	403 Forbidden, Not authenticated.	PASS*
TC12	GET s neplatným tokenem	Neplatný token je odmítnut.	401 Unauthorized, INVALID_TOKEN.	PASS
TC13	POST duplicitní e-mail	Duplicitní e-mail je odmítnut.	400 Bad Request, STUDENT_EMAIL_EXISTS.	PASS

TC14	POST bez e-mailu	Ověření validačního chování payloadu.	201 Created, ID 250, email = null.	OBS
------	------------------	---------------------------------------	------------------------------------	------------

* U TC11 je jako PASS hodnoceno to, že API přístup bez tokenu skutečně odmítlo. Pokud Swagger pro konkrétní GET výslovně požaduje 401, je vhodné status 403 evidovat jako samostatný defect.

Poznámka k odevzdání: požadavky, request body, očekávané a skutečné výsledky i SQL příkazy jsou uvedeny přímo jako text, aby byly snadno kontrolovatelné.

4. Průběh hlavního CRUD scénáře

4.1 Vytvoření studenta - POST

Byl vytvořen vlastní testovací student, aby bylo možné bezpečně provést následné GET, PUT a DELETE testy. API při vytvoření vrátilo 201 Created a přidělilo studentovi ID 249.

POST /students

```
{
  "firstName": "Test",
  "lastName": "Student",
  "email": "student.test.20260817.1642@example.com",
  "isEuCitizen": true,
  "zipCode": "30100",
  "numberOfFailedStudies": 0
}
```

4.2 SQL kontrola po vytvoření

Po vytvoření byl záznam ověřen pouze čtecím SQL dotazem:

```
SELECT *
FROM public.students
WHERE id = 249;
```

4.3 Aktualizace studenta - PUT

Při aktualizaci byly změněny hodnoty firstName a numberOfFailedStudies. Současně byla znovu odeslána hodnota isEuCitizen = true. API odpovědělo 200 OK, ale v odpovědi se isEuCitizen změnilo na false.

```
PUT /students/249
```

```
{
  "firstName": "TestUpdated",
  "lastName": "Student",
  "email": "student.test.20260817.1642@example.com",
  "isEuCitizen": true,
  "zipCode": "30100",
  "numberOfFailedStudies": 1
}
```

4.4 Ověření po PUT

Následný GET /students/249 vrátil 200 OK, firstName = TestUpdated, isEuCitizen = false a numberOfFailedStudies = 1. Stejná hodnota is_eu_citizen = false byla následně potvrzena čtecím SQL dotazem v databázi.

4.5 Odstranění studenta - DELETE

DELETE požadavek na studenta 249 vrátil 200 OK a zprávu, že student byl úspěšně odstraněn. Následné ověření však ukázalo, že záznam zůstal dostupný přes API i v databázi.

```
SELECT *
FROM public.students
WHERE id = 249;
```

5. Negativní a validační testy

5.1 Požadavek bez tokenu

GET bez Bearer tokenu byl odmítnut. API vrátilo 403 Forbidden s odpovědí Not authenticated. Bezpečnostní kontrola tedy funguje; přesnou konvenci status kódu je vhodné porovnat s dokumentací konkrétní metody.

5.2 Neplatný token

Při odeslání neplatné hodnoty tokenu API vrací 401 Unauthorized a kód INVALID_TOKEN.

5.3 Duplicitní e-mail

Při pokusu vytvořit dalšího studenta se stejným e-mailem API správně zabránilo duplicitě a vrátilo 400 Bad Request.

5.4 POST bez pole email - pozorování

Byl proveden doplňkový validační test, při kterém pole email nebylo v JSON payloadu uvedeno. API záznam přijalo, vrátilo 201 Created a vytvořilo studenta ID 250 s hodnotou email = null. Stejný stav byl potvrzen v databázi. Tento výsledek je v dokumentu veden jako pozorování, nikoli jako potvrzený defect, dokud nebude ve Swagger Schema ověřeno, že je email skutečně povinné pole.

6. Defect reporty

6.1 BUG-01 - PUT neuchová hodnotu isEuCitizen

ID	BUG-01
Název	PUT změny hodnotu isEuCitizen z true na false
Endpoint	PUT /students/249
Předpoklad	Existuje vlastní testovací student ID 249 a request je autorizovaný.
Kroky	1. Odeslat PUT s isEuCitizen=true. 2. Ověřit response. 3. Odeslat GET /students/249. 4. Ověřit záznam přes SELECT v DB.
Očekávaný výsledek	Hodnota isEuCitizen zůstane true v response, následném GET i databázi.
Skutečný výsledek	PUT vrátí 200 OK, ale response a GET obsahují isEuCitizen=false; databáze obsahuje is_eu_citizen=false.
Výsledek	FAIL
Závažnost	Střední - API mění hodnotu odeslaného atributu.
Důkaz	PUT request/response, následný GET a SQL SELECT nad studentem ID 249.

6.2 BUG-02 - DELETE hlásí úspěch, ale záznam nesmaže

ID	BUG-02
Název	DELETE vrátí úspěch, student ale zůstane dostupný
Endpoint	DELETE /students/249
Předpoklad	Existuje vlastní testovací student ID 249 a request je autorizovaný.

Kroky	1. Odeslat DELETE /students/249. 2. Ověřit response. 3. Odeslat GET /students/249. 4. Spustit SELECT v DB.
Očekávaný výsledek	Po úspěšném DELETE student již není dostupný přes GET a v DB není nalezen.
Skutečný výsledek	DELETE vrátí 200 OK a zprávu 'Student deleted successfully', ale GET vrací 200 OK se studentem a SQL stále vrací 1 řádek.
Výsledek	FAIL
Závažnost	Vysoká - základní CRUD operace DELETE fakticky neodstraní data.
Důkaz	DELETE response, následný GET a SQL SELECT nad studentem ID 249.

Pozorování k dalšímu ověření: POST bez emailu vytvořil ID 250 s email=NULL. Než bude zapsán jako defect, je potřeba v POST /students -> Schema potvrdit, zda je email povinné pole.

7. SQL kontroly

Databáze byla použita pouze ke čtení. Základní kontrolní dotaz:

```
SELECT *
FROM public.students
WHERE id = 249;
```

Stejný SELECT byl spuštěn po vytvoření, po PUT a po DELETE. Díky tomu bylo možné porovnat stav API se skutečným stavem dat v PostgreSQL databázi.

V databázi nebyly použity příkazy INSERT, UPDATE ani DELETE. Veškeré změny dat byly prováděny výhradně přes REST API.

8. Závěr

Hlavní CRUD scénář byl proveden od vytvoření vlastního studenta až po pokus o jeho odstranění. POST a GET fungovaly v základním pozitivním scénáři. SQL kontrola potvrdila vytvoření záznamu. Při PUT byl nalezen nesoulad hodnoty isEuCitizen a při DELETE kritičtější problém: API vrací úspěšnou odpověď, ale student zůstává dostupný přes GET a je stále uložen v databázi.

Negativní testy dále ověřily odmítnutí požadavku bez autorizace, odmítnutí neplatného tokenu a blokaci duplicitního e-mailu. Test bez e-mailu byl zachycen jako pozorování k ověření proti Swagger Schema.

Výsledek projektu: testování obsahuje pozitivní scénáře, negativní scénáře, SQL kontroly a dva potvrzené defect reporty.

9. Konečný kontrolní seznam

- ✓ POST /login otestován.
 - ✓ POST /students otestován.
 - ✓ GET /students/{student_id} otestován.
 - ✓ PUT /students/{student_id} otestován.
 - ✓ DELETE /students/{student_id} otestován.
 - ✓ SQL kontrola po POST provedena.
 - ✓ SQL kontrola po PUT provedena.
 - ✓ SQL kontrola po DELETE provedena.
 - ✓ Negativní test duplicitního e-mailu proveden.
 - ✓ Negativní test autorizace proveden.
 - ✓ BUG-01 zdokumentován.
 - ✓ BUG-02 zdokumentován.
- ✓ GitHub repozitář obsahuje PDF, README.md a sanitizovanou Postman collection.